

Startup Project Plan

Team Members

Ryan Hoo - u1511875

Connor Kolodziej - u1458335

Luke Stansbury - u1422313

GitHub Repo: [Repo](#)

Task Assignment:

For this stage of this assignment, our goal is to create a prototype/skeleton of the Android drawing application. We will start by establishing the main structure of the app, including the navigation and screens. We will also implement basic ui and drawing functionality while ensuring compatibility with both phone and tablet screens

In specific, we will start by implementing:

- Splash Screen
- Main Gallery Screen
- Drawing Selection/Creation Screen (potentially combined with gallery screen)
- Drawing/Editing Screen (only accessible under main screen)
- Backup/Sharing Screen (only accessible under main screen)
- Basic Freedraw
- At least 1 drawing tool (pen, brush, etc)
- Adjustable Brush Size
- RGB Color Selection (or at least some kind of color selection)
- Local Saving (import/export) of drawings
- Initial automated tests (more detail below)
- Unit tests
- AI object detection (detects what things are in a drawing/image)

Screen Navigation

The navigation flow for the application will be:

App Launch → Splash Screen → Main Gallery Screen

Under Main Gallery Screen:

New Drawing → Drawing Editor

Existing Drawing → Drawing Editor

Backup/Sharing → Backup Screen

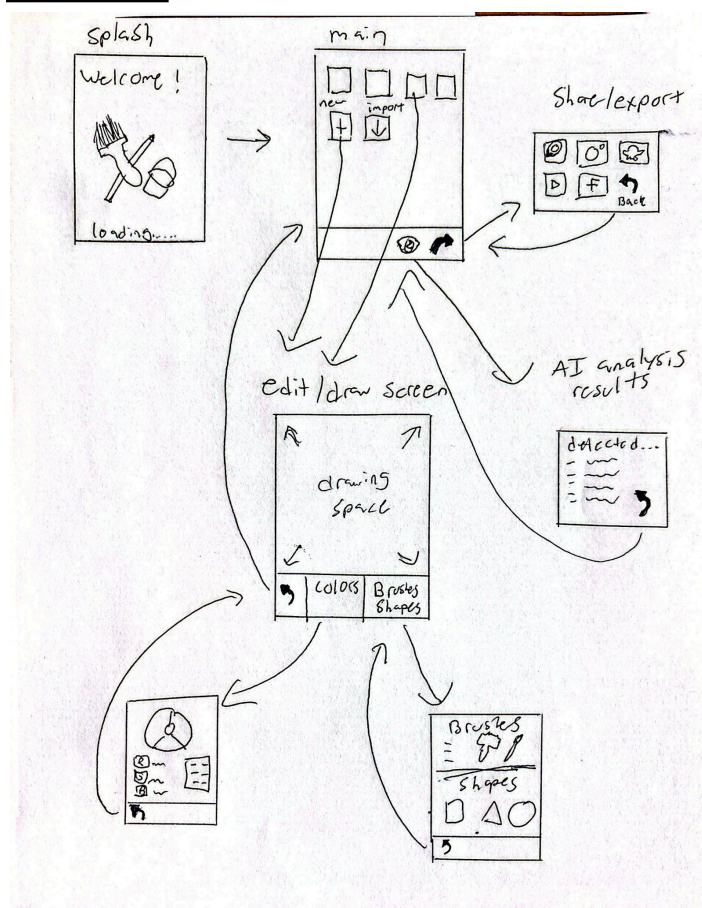
The splash screen will only appear briefly in the app launch before going to the main screen.

The gallery will display saved drawings and allow the user to either create a new drawing or open an existing one.

The drawing editor will have the drawing canvas and drawing controls and eventually AI integration.

The backup/sharing screen will eventually have sharing functionality.

Wireframes



- Splash: Displays welcome text, some indication of loading, and a logo. Once loading concludes, the user will be displayed the Main screen
- Main: Shows gallery of existing drawings with two extra squares: One for starting a new drawing, one for importing an existing one. It also contains a menu bar with options for sharing/exporting and AI analysis
 - Upon clicking/tapping the square for starting a new drawing, the user is displayed the Edit/Draw screen

- Upon clicking/tapping the square for importing an existing drawing, the device's native screen for image/file selection will be displayed. When an image/file is selected from said screen, and if what is selected is an image or drawing, the Edit/Draw Screen will be displayed with the selected image already rendered.
- Upon clicking the sharing/exporting button, the user is taken to the Share/Export subscreen
- Upon clicking the AI analysis button, the user is displayed two things on a new screen:
 - A list of AI interpretations of each drawing in the gallery
 - A button to return to the Main screen
- Share/Export: Sub-screen displaying options for sharing and exporting along with possible destination apps. Also shown is a button to return to the Edit/Draw main screen
- Edit/Draw: Consists of two main elements: a blank space for drawing, and a menu bar containing various sub-elements.
 - The menu bars sub-elements consists of the following:
 - A back button that brings a user back to the Main screen
 - A colors menu that, when pressed, displays a color wheel for choosing the color of the drawing tool. Also shown is a button to return to the Edit/Draw main screen.
 - A brush shapes menu that allows a user to select different brush styles and shapes. Also shown is a button to return to the Edit/Draw main screen.

Project Structure

This project will be separated into screens, drawing logic, drawing storage/data, and navigation.

```

app /
  data /
    DrawingData.kt
  ui/
    splash/
      SplashScreen.kt
    gallery/
      GalleryScreen.kt
      GalleryModel.kt
    editor/
      DrawingScreen.kt
      DrawingCanvas.kt

```

```
DrawingModel.kt
DrawingTool.kt
backup/
    BackupScreen.kt
navigation/
    AppNavigation.kt
MainActivity.kt
```

This structure may change as the project continues to develop.

Member/Tasks Breakdown

UI / Screen Logic - Luke Stansbury

Drawing / Editor Logic & Function - Connor Kolodziej

Code Management, AI Implementation, Testing - Ryan Hoo

Planned Classes/Functions

Some initial classes and functions may include:

Drawing

Represents a saved drawing.

Possible fields:

```
id
title
filePath
createdAt
modifiedAt
```

DrawingTool

Represents the currently selected drawing tool.

```
PEN
BRUSH
ERASER
LINE
RECTANGLE
CIRCLE
```

DrawingViewModel

Responsible for managing the drawing editor state.

Possible values/functions:

selectedTool
selectedColor
brushSize

setTool()
setColor()
setBrushSize()
undo()
redo()
saveDrawing()

DrawingRepository

Responsible for saving, loading, updating, and deleting drawings.

Possible functions:

createDrawing()
saveDrawing()
loadDrawings()
updateDrawing()
deleteDrawing()

These are not final and may change throughout the development of this project.

Automated Tests Plan

Drawing Repository Tests

- Creating a drawing adds it to storage.
- Saving a drawing updates the stored drawing.
- Deleting a drawing removes it.
- Loading drawings returns previously saved drawings.

Drawing Editor Tests

- Selecting a drawing tool updates the current tool.
- Changing brush size updates the selected brush size.
- Selecting a color updates the current drawing color.
- Undo removes the most recent drawing action.
- Redo restores a previously undone action.

Gallery Tests

- Saved drawings appear in the gallery.
- Opening a drawing navigates to the drawing editor.
- Creating a new drawing opens an empty drawing canvas.

Some functionality may not initially be covered by automated testing, including:

- Exact appearance of individual brush strokes
- Visual correctness of the RGB color wheel
- Behavior of every third-party application used for sharing
- Cloud synchronization
- AI object detection
- Device-specific UI differences

These areas may be tested manually or receive additional automated testing in later phases.

Git Version Control Plan

Development will use GitHub with separate feature branches.

Alongside the main branch, each member will work with different branches to ensure MVC Architecture.

For example, potential branches could include:

model/DrawingLogic
 model/LocalStorage
 view/Splash
 view/Gallery
 view/DrawingEditor
 view/BackupSharing
 view/ColorPicker
 controller/Navigation
 controller/DrawingTools
 controller/DrawingActions
 controller/Gallery
 controller/Sharing

Future Features

Future features both part of the project requirements and outside the requirements that may be implemented include:

Cloud Backup/Restore
 AI Integration
 Additional Drawing Tools

Additional Automated Tests

Improved UI/Tablet Layouts

Displaying detected objects & detection results